

Laboratorio 8 - Resumen y Posibles Preguntas para Interrogación

Curso: MDS7202 - Laboratorio de Programación Científica para Ciencia de Datos

Tema: Optimización de Hiperparámetros, MLflow y Deploy con FastAPI/Docker



RESUMEN EJECUTIVO DEL PROYECTO

Problema

Desarrollar un sistema completo de Machine Learning para predecir la **potabilidad del agua** (si es apta para consumo humano) basándose en 9 parámetros químicos.

Solución Implementada

Pipeline end-to-end de ML que incluye:

1. **Preprocesamiento:** Imputación de valores nulos con mediana y estandarización
 2. **Optimización:** Búsqueda de hiperparámetros con Optuna (50 trials)
 3. **Tracking:** Registro de experimentos con MLflow
 4. **Modelo:** XGBoost Classifier optimizado
 5. **Deploy:** API REST con FastAPI
 6. **Containerización:** Docker para portabilidad
-



OBJETIVOS DEL LABORATORIO

1. ☒ Implementar optimización de hiperparámetros con **Optuna**
 2. ☒ Registrar experimentos y modelos con **MLflow**
 3. ☒ Crear API REST para servir predicciones con **FastAPI**
 4. ☒ Containerizar la aplicación con **Docker**
 5. ☒ Aplicar buenas prácticas de MLOps
-



COMPONENTES TÉCNICOS PRINCIPALES

1. optimize.py - Pipeline de Entrenamiento

Funcionalidades:

- Carga y preprocesamiento de datos (water_potability.csv)
- Imputación de valores nulos con mediana
- Split train/test (80/20) con estratificación
- Estandarización con StandardScaler
- Optimización con Optuna (50 trials)
- Cross-validation (5 folds) con métrica F1-score

- Logging automático en MLflow de todos los trials
- Generación de visualizaciones (Optuna plots + feature importance)
- Guardado de artefactos: modelo .pkl, config JSON, requirements.txt

Hiperparámetros Optimizados:



json

```
{
  "n_estimators": 750,
  "learning_rate": 0.294,
  "max_depth": 9,
  "subsample": 0.755,
  "colsample_bytree": 0.898,
  "gamma": 0.251,
  "reg_alpha": 1.667,
  "reg_lambda": 3.600
}
```

Espacio de búsqueda:

- n_estimators: [100, 1000] (step=50)
- learning_rate: [0.01, 0.3] (escala logarítmica)
- max_depth: [3, 10]
- subsample: [0.6, 1.0]
- colsample_bytree: [0.6, 1.0]
- gamma: [0, 5]
- reg_alpha: [0, 5]
- reg_lambda: [0, 5]

2. main.py - API de Predicción

Endpoints:

- GET / - Descripción del proyecto y API
- POST /potabilidad/ - Recibe 9 features y retorna predicción

Características:

- Validación de entrada con Pydantic (WaterFeatures model)
- Carga automática del mejor modelo desde MLflow
- Re-entrenamiento del scaler usando mismo split que optimize.py
- Manejo de errores (503, 500)
- Documentación interactiva automática en /docs

Input esperado (JSON):



json

```
{
  "ph": 10.316,
  "Hardness": 217.266,
  "Solids": 10676.508,
  "Chloramines": 3.445,
  "Sulfate": 397.754,
  "Conductivity": 492.206,
  "Organic_carbon": 12.812,
  "Trihalomethanes": 72.281,
  "Turbidity": 3.407
}
```

Output:



json

```
{
  "potabilidad": 1 // 1=potable, 0=no potable
}
```

3. Dockerfile - Containerización

Estrategia:

- Imagen base: python:3.9-slim (ligera)
- Instalación de dependencias (cacheado)
- Copia solo del código necesario (main.py)
- Exposición del puerto 8000
- CMD con uvicorn para servir la API

Comandos Docker:



bash

Build

`docker build -t water-api .`

Run

`docker run -p 8000:8000 water-api`

4. MLflow Tracking

Estructura de experimentos:

- Experimento principal: "Optimización Potabilidad Agua XGBoost"
- 50 runs anidados (uno por trial de Optuna)
- 1 run final de reporte con todos los artefactos

Artefactos guardados en run final:

- `/plots/` → 4 gráficos (optuna_history, param_importances, slice, feature_importance)
- `/models/` → `best_potability_model.pkl`
- `best_model_config.json` → hiperparámetros del mejor modelo
- `requirements.txt` → versiones de librerías

ANÁLISIS DE RESULTADOS

Feature Importance (según XGBoost)

Top 3 variables más importantes:

1. **Solids** (673.0) - Sólidos totales disueltos
2. **Conductivity** (659.0) - Conductividad eléctrica
3. **Chloramines** (639.0) - Nivel de cloraminas

Menos importante:

- **Organic_carbon** (575.0)

Optimización Optuna

- **Mejor F1-score (CV):** ~0.50 (visible en optimization history plot)
- **Hiperparámetro más importante:** Gamma (0.80 de importancia)
- **Convergencia:** El modelo mejora consistentemente hasta ~trial 40

POSIBLES PREGUNTAS DE INTERROGACIÓN

A. PREGUNTAS CONCEPTUALES

1. MLOps y Tracking de Experimentos

P: ¿Qué es MLflow y para qué sirve en este proyecto? R: MLflow es una plataforma open-source para gestionar el ciclo de vida de ML. En el proyecto se usa para:

- Tracking de experimentos (registrar parámetros, métricas)
- Almacenar artefactos (modelos, plots, configs)
- Versionamiento de modelos
- Facilitar reproducibilidad
- Comparar diferentes trials de optimización

P: ¿Qué son los "runs" y "experiments" en MLflow? R:

- **Experiment:** Contenedor lógico que agrupa runs relacionados (e.g., "Optimización Potabilidad Agua")
- **Run:** Ejecución individual con parámetros, métricas y artefactos específicos (e.g., cada trial de Optuna)
- En el proyecto: 1 experiment, 51 runs (50 trials + 1 reporte final)

P: ¿Qué es "nested run" en MLflow? R: Es un run hijo dentro de otro run padre. En el código, cada trial de Optuna es un nested run (nested=True) que se agrupa bajo un run principal, facilitando la organización y visualización.

2. Optimización de Hiperparámetros

P: ¿Por qué usar Optuna en lugar de GridSearch o RandomSearch? R:

- **GridSearch:** Exhaustivo pero muy lento (prueba TODAS las combinaciones)
- **RandomSearch:** Más rápido pero aleatorio, puede perder buenos valores
- **Optuna:** Usa algoritmos Bayesianos (TPE - Tree-structured Parzen Estimator) que aprenden de trials anteriores para sugerir mejores hiperparámetros, siendo más eficiente que RandomSearch y mucho más rápido que GridSearch

P: ¿Qué es la "función objetivo" en Optuna? R: Es la función que Optuna intenta optimizar (maximizar o minimizar). En el proyecto:



python

```
def objective(trial, X_train, y_train) -> float:
    # ... define parámetros, entrena modelo, evalúa
    return f1_score_mean # <-- Esto es lo que Optuna maximiza
```

P: ¿Por qué se usa F1-score como métrica objetivo? R:

- El dataset de potabilidad es **desbalanceado** (no hay igual cantidad de muestras potables/no potables)
- F1-score es la media armónica de precision y recall, balanceando falsos positivos y falsos negativos
- Accuracy sería engañosa en datasets desbalanceados (podría predecir siempre la clase mayoritaria)

P: ¿Qué diferencia hay entre "eval_metric" (logloss) y la métrica objetivo (F1)? R:

- **eval_metric='logloss':** Métrica interna de XGBoost para entrenar (optimiza la probabilidad)
- **F1-score:** Métrica externa para Optuna (evalúa clasificación binaria balanceada)
- Logloss optimiza probabilidades, F1 optimiza la frontera de decisión

P: ¿Por qué usar log=True en learning_rate? R: Porque el learning rate funciona mejor en escala logarítmica. Un cambio de 0.01 a 0.02 (100% de incremento) es más significativo que de 0.2 a 0.21 (5% de incremento). log=True muestrea uniformemente en escala log.

3. XGBoost y Gradient Boosting

P: ¿Qué es XGBoost y por qué es efectivo? R: XGBoost (eXtreme Gradient Boosting) es un algoritmo de ensemble que:

- Construye árboles de decisión de forma secuencial
- Cada árbol corrige errores del anterior
- Usa regularización (L1/L2) para evitar overfitting
- Es muy eficiente y paralelizable
- Maneja bien datos tabulares y valores faltantes

P: Explica los hiperparámetros principales optimizados: R:

- **n_estimators:** Número de árboles (más árboles = más capacidad, pero riesgo de overfitting)
- **learning_rate:** Tasa de aprendizaje (más bajo = más conservador, necesita más árboles)
- **max_depth:** Profundidad máxima del árbol (mayor = más complejo)
- **subsample:** Fracción de datos para entrenar cada árbol (< 1 previene overfitting)
- **colsample_bytree:** Fracción de features por árbol (como Random Forest)
- **gamma:** Reducción mínima de loss para hacer un split (regularización)
- **reg_alpha (L1):** Regularización Lasso en pesos de hojas
- **reg_lambda (L2):** Regularización Ridge en pesos de hojas

P: ¿Qué significa "gamma: 0.251" en los resultados? R: Gamma es un umbral de regularización. Un split solo se hace si reduce el loss en al menos 0.251. Valores más altos hacen el modelo más conservador (menos propenso a overfitting).

4. Preprocesamiento

P: ¿Por qué imputar valores nulos con la mediana y no la media? R:

- La mediana es **robusta a outliers** (no se ve afectada por valores extremos)
- En datos químicos del agua pueden haber mediciones anómalas
- La media podría distorsionarse con valores atípicos

P: ¿Por qué es importante usar StandardScaler? R:

- Las features tienen diferentes escalas (pH: 0-14, Solids: miles)
- XGBoost es basado en árboles, NO requiere escalamiento técnicamente
- Sin embargo, se escala por buenas prácticas y porque puede ayudar a la convergencia
- También facilita la interpretación y comparación de features

P: ¿Por qué es crítico usar stratify=y en train_test_split? R: Para mantener la misma proporción de clases (potable/no potable) en train y test. Si hay 60% no potable / 40% potable en el dataset completo, ambos subsets mantendrán esta proporción, evitando sesgo en la evaluación.

P: ¿Por qué es importante que main.py use el MISMO split que optimize.py para el scaler? R: Porque el scaler debe aprender las estadísticas (media, std) del MISMO conjunto de entrenamiento. Si usara datos diferentes, las transformaciones no serían consistentes y las predicciones serían incorrectas.

5. API y Deployment

P: ¿Qué es FastAPI y por qué usarla? R: FastAPI es un framework moderno para crear APIs REST en Python:

- Muy rápido (basado en Starlette y Pydantic)
- Documentación automática (Swagger UI en /docs)
- Validación automática de datos con Pydantic
- Type hints nativos de Python
- Async/await support

P: ¿Qué hace Pydantic en el código? R: Pydantic crea modelos de datos con validación automática:



python

```
class WaterFeatures(BaseModel):
    ph: float # <- Valida que sea float
    Hardness: float
    # ... etc
```

Si el cliente envía {"ph": "texto"}, FastAPI rechaza automáticamente con error 422.

P: ¿Por qué cargar el modelo UNA VEZ al inicio y no en cada request? R:

- Cargar el modelo desde MLflow es lento (~segundos)
- Si se cargara en cada request, la API sería extremadamente lenta
- Se carga al inicio (cuando arranca uvicorn) y se mantiene en memoria
- **Trade-off:** Usa más memoria, pero es MUCHO más rápido

P: ¿Qué es uvicorn? R: Es un servidor ASGI (Asynchronous Server Gateway Interface) ultrarrápido para servir aplicaciones async de Python como FastAPI. Es el equivalente a gunicorn pero para aplicaciones asíncronas.

6. Docker y Containerización

P: ¿Por qué usar Docker? R:

- **Portabilidad:** "Funciona en mi máquina" → "Funciona en todas las máquinas"
- **Reproducibilidad:** Mismo ambiente en desarrollo, test, producción
- **Aislamiento:** No contamina el sistema host
- **Fácil deploy:** Cloud providers aceptan imágenes Docker directamente

P: ¿Qué hace cada línea del Dockerfile? R:

1. FROM python:3.9-slim → Imagen base ligera de Python
2. WORKDIR /app → Establece directorio de trabajo
3. COPY requirements.txt . → Copia solo dependencias primero (para cache)
4. RUN pip install ... → Instala dependencias (se cachea si requirements no cambia)
5. COPY main.py . → Copia código de la app
6. EXPOSE 8000 → Documenta que usa puerto 8000
7. CMD [...] → Comando para ejecutar cuando inicia el contenedor

P: ¿Por qué copiar requirements.txt ANTES que main.py? R: Para aprovechar el **layer caching** de Docker. Las dependencias casi nunca cambian, pero el código sí. Si copiamos todo junto, cada cambio en main.py reinstalaría todas las dependencias (lento). Separándolos, Docker reutiliza la capa de dependencias si requirements.txt no cambió.

P: ¿Qué significa --host 0.0.0.0 en el CMD? R:

- 127.0.0.1 → Solo acepta conexiones locales (dentro del contenedor)
 - 0.0.0.0 → Acepta conexiones desde cualquier IP (necesario para acceder desde fuera del contenedor)
 - Sin esto, no podrías hacer localhost:8000 desde tu host
-

B. PREGUNTAS TÉCNICAS DE IMPLEMENTACIÓN

7. Sobre el Código

P: ¿Por qué se usa `mlflow.xgboost.autolog()`? R: Autolog automáticamente registra:

- Parámetros del modelo
- Métricas de entrenamiento
- El modelo serializado como artefacto
- Signature del modelo (input/output schema) Sin autolog, habría que hacer `mlflow.log_param()`, `mlflow.log_metric()`, etc. manualmente.

P: ¿Qué pasa si no se encuentra el archivo `water_potability.csv`? R:

- `load_preprocess_data()` retorna `None` para todas las variables
- `optimize_model()` detecta esto y termina con `return None`
- El programa NO crashea, imprime error y sale gracefully

P: En `main.py`, ¿cómo se busca el mejor modelo en MLflow? R:



python

1. Busca el run con nombre específico

```
filter_string="tags.mlflow.runName = 'Mejor_Modelo_y_Reporte_Final'"
```

2. Descarga el artefacto .pkl desde ese run

```
model_uri = f"runs://{best_run_id}/models/best_potability_model.pkl"
local_model_path = mlflow.artifacts.download_artifacts(artifact_uri=model_uri)
```

3. Carga el pickle

```
with open(local_model_path, "rb") as f:
    model = pickle.load(f)
```

P: ¿Por qué se usa `cross_val_score` en la función objetivo? R: Para evaluar el modelo de forma más robusta:

- Divide train en 5 folds
- Entrena 5 modelos (cada uno usa 4 folds para train, 1 para validación)
- Retorna el promedio de F1-score de los 5 modelos
- Reduce la varianza de la evaluación (más confiable que un solo split)

P: ¿Qué hace `n_jobs=-1` en `cross_val_score`? R: Usa todos los cores de CPU disponibles para paralelizar el cross-validation (entrena los 5 folds en paralelo). Acelera mucho el proceso.

P: ¿Por qué se eliminan los archivos locales al final de `optimize.py`? R:



python


```
for f in local_files_to_clean:
    if os.path.exists(f):
        os.remove(f)
```

Porque ya fueron guardados en MLflow como artefactos. Dejarlos en el filesystem local sería redundante y ocuparía espacio innecesario.

P: ¿Qué pasa si el modelo retorna una predicción con más de un elemento? R:



python

```
prediction = int(prediction_array[0])
```

Se extrae solo el primer elemento y se convierte a int. XGBoost.predict() retorna un numpy array, aunque sea de un solo elemento.

8. Cross-Validation y Evaluación

P: ¿Por qué usar CV=5 y no más? R:

- CV=5 es un buen balance entre sesgo y varianza
- CV=10 sería más preciso pero 2x más lento
- CV=3 sería más rápido pero menos confiable
- Con 50 trials, usar CV=10 haría la optimización muy lenta

P: ¿Cuál es la diferencia entre el F1-score de CV y el del test set? R:

- **CV F1-score:** Estimación de rendimiento en datos no vistos (promedio de 5 folds en train)
- **Test F1-score:** Evaluación final en datos completamente separados
- El test set solo se evalúa UNA VEZ al final (nunca se usa en optimización)

P: ¿Por qué no se reporta el test accuracy en el código? R: Porque el focus está en la optimización (que usa CV). El test set debería evaluarse al final del proyecto, pero el código se enfoca en encontrar el mejor modelo, no en reportar métricas finales. En un proyecto real, se agregaría:



python

```
y_pred = best_model.predict(X_test)
test_f1 = f1_score(y_test, y_pred)
```

C. PREGUNTAS DE ANÁLISIS Y RESULTADOS

9. Interpretación de Gráficos

P: ¿Qué nos dice el gráfico de Optimization History? R:

- **Eje X:** Número de trial (0-50)
- **Eje Y:** F1-score alcanzado
- **Puntos azules:** F1-score de cada trial
- **Línea roja:** Mejor F1-score hasta ese momento
- **Interpretación:** El modelo mejora hasta ~trial 40, luego se estabiliza (~0.50)

P: ¿Qué significa que gamma tenga importancia 0.80 en Hyperparameter Importances? R:

- Gamma es el hiperparámetro que más afecta el F1-score final
- Cambios en gamma producen cambios grandes en performance
- Los demás parámetros tienen mucho menos impacto
- **Implicación:** Si tuviéramos tiempo limitado, deberíamos enfocarnos en afinar gamma

P: ¿Qué muestra el Slice Plot? R: Muestra la relación entre cada hiperparámetro individual y el F1-score:

- Cada subgráfico es un parámetro
- Puntos azules son trials (color indica orden temporal)
- Permite ver tendencias (e.g., F1 más alto con gamma cercano a 0)

P: Según Feature Importance, ¿cuáles son las variables más relevantes para predecir potabilidad? R:

1. Solids (673.0) → Sólidos disueltos totales
2. Conductivity (659.0) → Conductividad eléctrica
3. Chloramines (639.0) → Nivel de cloraminas

Estas tres variables son las que más influyen en las decisiones del modelo.

10. Mejoras y Limitaciones

P: ¿Qué mejoras harías al proyecto? R:

1. **Métricas adicionales:** Agregar precision, recall, ROC-AUC
2. **Test set evaluation:** Reportar métricas en test set
3. **Model monitoring:** Agregar logging de predicciones para detectar drift
4. **CI/CD:** Automatizar testing y deploy
5. **Manejo de outliers:** Analizar y tratar valores extremos antes de imputar
6. **Feature engineering:** Crear interacciones o transformaciones de features
7. **Ensemble:** Probar stacking con otros modelos (RF, LightGBM)
8. **A/B testing:** Comparar modelos en producción

P: ¿Cuáles son las limitaciones del proyecto actual? R:

1. No se evalúa en test set (solo CV)
2. No hay manejo de concept drift
3. No hay monitoring de predicciones
4. Falta análisis exploratorio de datos
5. No hay manejo de outliers explícito
6. El scaler se re-crea en main.py (debería guardarse como artefacto)
7. No hay versionamiento de API
8. Falta manejo de concurrencia en producción

P: ¿Por qué el F1-score es ~0.50 y no más alto? R: Posibles razones:

1. **Dataset desbalanceado:** Si hay muchas más muestras de una clase

2. **Features poco informativas:** Las variables químicas pueden no ser suficientes
3. **Ruido en los datos:** Mediciones químicas con error
4. **Complejidad del problema:** Potabilidad puede depender de factores no capturados
5. **Necesidad de feature engineering:** Interacciones entre variables

P: ¿Cómo escalarías este servicio para producción? R:

1. **Horizontal scaling:** Múltiples instancias de la API con load balancer
 2. **Caching:** Redis para predicciones frecuentes
 3. **Async predictions:** Queue system (Celery + RabbitMQ) para batch predictions
 4. **Database:** PostgreSQL para logging de requests/responses
 5. **Monitoring:** Prometheus + Grafana para métricas
 6. **Logging:** ELK stack (Elasticsearch, Logstash, Kibana)
 7. **Authentication:** OAuth2/JWT para seguridad
 8. **Rate limiting:** Prevenir abuse
-

D. PREGUNTAS DE INTEGRACIÓN

11. Flujo Completo

P: Describe el flujo completo desde datos hasta predicción: R:

1. **Entrenamiento (optimize.py):**
 - Carga water_potability.csv
 - Imputa nulos con mediana
 - Split train/test (80/20, stratified)
 - Escala con StandardScaler
 - Optuna ejecuta 50 trials con CV
 - Cada trial se registra en MLflow
 - Se guarda el mejor modelo y artefactos
2. **Deploy (main.py + Docker):**
 - Se construye imagen Docker con dependencias
 - Al iniciar, carga el mejor modelo desde MLflow
 - Re-crea el scaler con mismo train set
 - Levanta API en puerto 8000
3. **Predicción:**
 - Cliente envía POST con 9 features
 - Pydantic valida los datos
 - Se escalan las features
 - Se predice con el modelo
 - Se retorna {potabilidad: 0 o 1}

P: ¿Qué pasaría si ejecutas main.py sin haber corrido optimize.py antes? R: El programa fallaría en `get_model()`:



python

```
if experiment is None:
    raise Exception(f'Experimento '{EXPERIMENT_NAME}' no encontrado. Ejecute optimize.py primero.")
```

Imprime error claro indicando que falta el experimento y termina.

P: ¿Cómo te aseguras de que la API use el mismo preprocesamiento que el entrenamiento? R:

- 1. **Mismo orden de features:** FEATURE_NAMES definido en ambos scripts
 - 2. **Mismo scaler:** get_fitted_scaler() replica el proceso de optimize.py:
 - Lee el mismo CSV
 - Mismo fillna con mediana
 - Mismo train_test_split con random_state=42
 - Fit del scaler en el mismo X_train
 - 3. **Documentación:** Los scripts tienen comentarios explicando esto
-

E. PREGUNTAS AVANZADAS / TRICKY

12. Debugging y Troubleshooting

P: El contenedor Docker arranca pero no puedes acceder a localhost:8000. ¿Por qué? R: Posibles causas:

- 1. No mapeaste el puerto: Falta -p 8000:8000 en docker run
- 2. El CMD usa 127.0.0.1 en vez de 0.0.0.0
- 3. Firewall bloqueando el puerto
- 4. El contenedor crasheó (verificar con docker logs <container_id>)

P: Las predicciones de la API son muy diferentes a las del notebook. ¿Qué revisarías? R:

- 1. **Scaler diferente:** ¿Usó los mismos datos de train?
- 2. **Modelo equivocado:** ¿Cargó el modelo correcto desde MLflow?
- 3. **Orden de features:** ¿Las columnas están en el mismo orden?
- 4. **Versión de librerías:** ¿sklearn o xgboost tienen versiones diferentes?
- 5. **Random state:** ¿El split de train/test es el mismo?

P: Optuna tarda mucho. ¿Cómo acelerarlo? R:

- 1. **Reducir CV folds:** cv=3 en vez de cv=5
- 2. **Reducir n_trials:** 20 en vez de 50
- 3. **Paralelizar:** Usar n_jobs=-1 en cross_val_score (ya está)
- 4. **Usar TPU/GPU:** XGBoost soporta tree_method='gpu_hist'
- 5. **Reducir n_estimators máximo:** 500 en vez de 1000
- 6. **Pruning:** Usar Optuna pruners para detener trials malos early

P: ¿Cómo debuggearías si MLflow no encuentra el experimento? R:

- 1. Verificar que estás en el directorio correcto (que contiene mlruns/)
 - 2. Comprobar que optimize.py se ejecutó correctamente
 - 3. Revisar que EXPERIMENT_NAME sea exactamente el mismo en ambos scripts
 - 4. Usar mlflow ui para verificar visualmente los experimentos
 - 5. Revisar permisos de la carpeta mlruns/
-

F. PREGUNTAS DE COMPARACIÓN

13. Alternativas y Trade-offs

P: XGBoost vs Random Forest vs Neural Networks para este problema: R:

- **XGBoost:** Mejor para datos tabulares, rápido, interpretable, maneja features de forma automática

- **Random Forest:** Similar a XGBoost pero más lento, menos overfitting
- **Neural Networks:** Overkill para dataset pequeño, necesita más datos, menos interpretable
- **Conclusión:** XGBoost es la mejor opción para este caso

P: Optuna vs Hyperopt vs Scikit-learn's RandomizedSearchCV: R:

- **Optuna:** Moderno, fácil de usar, buena visualización, buen algoritmo (TPE)
- **Hyperopt:** Más viejo, menos user-friendly, pero robusto
- **RandomizedSearchCV:** Simple, no requiere librería extra, pero menos eficiente (random)
- **GridSearchCV:** Exhaustivo pero extremadamente lento
- **Conclusión:** Optuna ofrece mejor balance eficiencia/usabilidad

P: FastAPI vs Flask: R:

- **FastAPI:**
 - ☒ Documentación automática
 - ☒ Validación con Pydantic
 - ☒ Más rápido (async)
 - ☒ Type hints nativos
 - ☒ Menos maduro, menos recursos online
- **Flask:**
 - ☒ Más establecido, mucha documentación
 - ☒ Más flexible (menos opinionado)
 - ☒ Sin validación automática
 - ☒ Sin documentación auto
 - ☒ Más lento (sync)

P: Pickle vs Joblib vs ONNX para guardar modelos: R:

- **Pickle:** Estándar de Python, fácil, pero no portable entre versiones
- **Joblib:** Más eficiente para arrays grandes (sklearn lo usa), pero similar a pickle
- **ONNX:** Portable entre frameworks (Python, C++, JS), más complejo de implementar
- **Para este proyecto:** Pickle está bien (todo en Python), pero Joblib sería mejor práctica

TIPS PARA LA INTERROGACIÓN

1. Sé específico con los números

- Mejor F1-score: ~0.50
- Trials: 50
- CV folds: 5
- Split: 80/20
- Random state: 42
- Features: 9

2. Entiende el "por qué"

No solo memorices QUÉ hace el código, sino POR QUÉ se eligió esa solución.

3. Prepara diagramas mentales

- Flujo de datos (CSV → Preprocessing → Model → API → Prediction)

- Arquitectura (optimize.py ↔ MLflow ↔ main.py ↔ Docker)

4. Conoce las limitaciones

Los profesores valoran que reconozcas qué se podría mejorar.

5. Practica explicar en voz alta

Simula que le explicas el proyecto a alguien que no sabe de ML.

CONCEPTOS CLAVE PARA REPASAR

1.  **Gradient Boosting** (cómo funciona XGBoost)
2.  **Bayesian Optimization** (TPE en Optuna)
3.  **Cross-Validation** (K-fold, stratified)
4.  **Imbalanced Classification** (F1-score, stratify)
5.  **StandardScaler** (z-score normalization)
6.  **RESTful APIs** (GET/POST, JSON, status codes)
7.  **Docker layers** (caching, multistage builds)
8.  **MLflow artifacts** (models, plots, configs)
9.  **Pydantic validation** (data schemas)
10.  **Nested runs** (parent/child runs en MLflow)

PREGUNTAS "KILLER" (MÁS DIFÍCILES)

P: Si tuvieras que deployar esto en AWS, ¿qué servicios usarías y por qué? R:

1. **ECS/EKS:** Para correr contenedores Docker (escalable)
2. **ECR:** Registro de imágenes Docker
3. **Application Load Balancer:** Distribuir tráfico
4. **RDS:** PostgreSQL para logging
5. **S3:** Almacenar artefactos de MLflow
6. **CloudWatch:** Monitoring y logging
7. **SageMaker:** Alternativa managed para ML (pero más caro)

P: ¿Cómo implementarías A/B testing entre dos versiones del modelo? R:

1. Deployar dos versiones de la API (v1, v2)
2. Load balancer que rutee X% del tráfico a v1 y Y% a v2
3. Logging de predicciones y feedback real
4. Calcular métricas en ambos (accuracy, latencia)
5. Statistical testing (t-test, chi-square) para determinar ganador
6. Gradual rollout del modelo ganador

P: ¿Cómo detectarías data drift en producción? R:

1. **Statistical tests:** Kolmogorov-Smirnov test en distribución de features
2. **Model performance:** Monitorear F1-score en tiempo real (si hay labels)
3. **Prediction distribution:** Shift en proporción de predicciones 0/1
4. **Feature statistics:** Monitorear media/std de cada feature

5. **Tools:** Evidently AI, WhyLabs, Fiddler

P: El modelo se entrenó con datos de 2020. Es 2025 y las predicciones son malas. ¿Qué harías? R:

1. **Análisis de drift:** Comparar distribuciones de features 2020 vs 2025
 2. **Retraining:** Re-entrenar con datos más recientes
 3. **Feature engineering:** Agregar features temporales (año, tendencias)
 4. **Ensemble:** Combinar modelo viejo + nuevo con pesos adaptativos
 5. **Active learning:** Pedir labels en muestras donde el modelo tiene baja confianza
 6. **Causality analysis:** Investigar qué cambió (normativas, tecnología, etc.)
-



CHECKLIST FINAL ANTES DE LA INTERROGACIÓN

- ☐ Puedo explicar el flujo completo sin mirar código
 - ☐ Entiendo cada hiperparámetro de XGBoost
 - ☐ Sé por qué se usa cada librería (Optuna, MLflow, FastAPI, Docker)
 - ☐ Puedo interpretar los 4 gráficos (Optuna + feature importance)
 - ☐ Conozco las limitaciones y mejoras posibles
 - ☐ Puedo debuggear errores comunes
 - ☐ Entiendo las decisiones de diseño (por qué X y no Y)
 - ☐ Sé responder preguntas de escalabilidad y producción
-



RECURSOS PARA PROFUNDIZAR

1. **Optuna Documentation:** <https://optuna.readthedocs.io/>
 2. **MLflow Tracking:** <https://mlflow.org/docs/latest/tracking.html>
 3. **XGBoost Parameters:** <https://xgboost.readthedocs.io/en/stable/parameter.html>
 4. **FastAPI Tutorial:** <https://fastapi.tiangolo.com/tutorial/>
 5. **Docker Best Practices:** <https://docs.docker.com/develop/dev-best-practices/>
-

¡Éxito en tu interrogación! 🚀